# Part 1 Image Classification

In [1]:

```
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Preprocessing steps required by assignment
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=(0.4467, 0.4398, 0.4066),
        std=(0.2241, 0.2215, 0.2239)
    )
])

# Load STL-10 dataset
train_dataset = datasets.STL10(
    root="./data",
    split="train",
    download=True,
    transform=transform
)

test_dataset = datasets.STL10(
    root="./data",
    split="test",
    download=True,
    transform=transform
)

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

print("STL-10 Preprocessing Complete")
print("Number of training samples:", len(train_dataset))
print("Number of testing samples:", len(test_dataset))
```

```
100.0%
STL-10 Preprocessing Complete
Number of training samples: 5000
Number of testing samples: 8000
```

ANN Architecture

In [2]:

```
import torch.nn as nn
import torch.optim as optim

class ANN(nn.Module):
    def __init__(self):
        super(ANN, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(12288, 1024),
```

```
            nn.ReLU(),
            nn.Linear(1024, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)  # Flatten
        return self.model(x)
```

In [3]:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

ann_model = ANN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(ann_model.parameters(), lr=0.001)

ann_losses = []

for epoch in range(5):
    ann_model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = ann_model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)
    ann_losses.append(epoch_loss)
    print(f"Epoch {epoch+1}/5 - ANN Loss: {epoch_loss:.4f}")
```

```
Epoch 1/5 - ANN Loss: 2.0672
Epoch 2/5 - ANN Loss: 1.6377
Epoch 3/5 - ANN Loss: 1.5346
Epoch 4/5 - ANN Loss: 1.3676
Epoch 5/5 - ANN Loss: 1.2275
```

In [ ]:

```
ann_model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = ann_model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

ann_accuracy = 100 * correct / total
print(f"ANN Test Accuracy: {ann_accuracy:.2f}%")
```

```
ANN Test Accuracy: 37.51%
```

CNN Architecture

In [5]:

```
import torch.nn as nn
import torch.optim as optim

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.MaxPool2d(kernel_size=2, stride=2)
        )

        # After convs:
        # 3x64x64 -> 32x64x64 -> pool -> 32x32x32
        # -> 64x32x32 -> 128x32x32 -> pool -> 128x16x16
        # 128 * 16 * 16 = 32768
        self.classifier = nn.Sequential(
            nn.Linear(32768, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)  # flatten
        x = self.classifier(x)
        return x
```

In [6]:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

cnn_model = CNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(cnn_model.parameters(), lr=0.0001)

cnn_losses = []

for epoch in range(15):
    cnn_model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
```

```
        optimizer.zero_grad()
        outputs = cnn_model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)
    cnn_losses.append(epoch_loss)
    print(f"Epoch {epoch+1}/15 - CNN Loss: {epoch_loss:.4f}")
```

```
Using device: cpu
Epoch 1/15 - CNN Loss: 1.8593
Epoch 2/15 - CNN Loss: 1.4954
Epoch 3/15 - CNN Loss: 1.3225
Epoch 4/15 - CNN Loss: 1.2028
Epoch 5/15 - CNN Loss: 1.0908
Epoch 6/15 - CNN Loss: 1.0208
Epoch 7/15 - CNN Loss: 0.9273
Epoch 8/15 - CNN Loss: 0.8367
Epoch 9/15 - CNN Loss: 0.7519
Epoch 10/15 - CNN Loss: 0.7056
Epoch 11/15 - CNN Loss: 0.6167
Epoch 12/15 - CNN Loss: 0.5701
Epoch 13/15 - CNN Loss: 0.4671
Epoch 14/15 - CNN Loss: 0.4203
Epoch 15/15 - CNN Loss: 0.3436
```

In [7]:

```
cnn_model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = cnn_model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

cnn_accuracy = 100 * correct / total
print(f"CNN Test Accuracy: {cnn_accuracy:.2f}%")
```

```
CNN Test Accuracy: 54.94%
```

## Comparison of ANN vs CNN Performance

The Artificial Neural Network (ANN) achieved a test accuracy of 37.51 percent on the STL-10 dataset, while the Convolutional Neural Network (CNN) achieved a significantly higher accuracy of 54.94 percent. This shows that the CNN was substantially more effective at learning meaningful patterns from the image data.

The ANN struggled primarily because it required the 64×64 RGB images to be flattened into a single vector of 12,288 values. This flattening process removes all spatial information, meaning the model cannot recognize relationships between neighboring pixels. As a result, the ANN treats each pixel independently and fails to capture important visual structures such as edges, shapes, and textures that define objects.

The CNN performed better because it preserved the spatial structure of the images and used convolutional layers to extract hierarchical features. The early convolution layers learned basic patterns such as edges and corners, while deeper layers learned more complex representations like object parts. Max pooling reduced noise and improved translation invariance, further improving classification performance.

The architectural components that made the biggest difference were the convolutional layers and pooling layers. These allowed the CNN to perform localized feature detection and progressively build higher-level representations, which is essential for image recognition tasks. In contrast, the ANN lacked any mechanism to exploit spatial relationships, resulting in weaker generalization and lower accuracy.

# Part 2 NLP + LLM

## Question 4: Preprocessing Steps in NLP

Preprocessing in Natural Language Processing (NLP) involves preparing raw text so it can be effectively analyzed by computational models. Common preprocessing steps include tokenization, which splits text into smaller units such as words or subwords, and lowercasing, which ensures consistency by converting all text to a uniform case. Stop-word removal may be applied to eliminate frequently occurring but low-information words such as “the” or “and.” Stemming and lemmatization reduce words to their base or root forms to minimize variation. Additional steps include removing punctuation, handling special characters, correcting spelling, and normalizing text formats. These processes reduce noise, standardize the data, and improve the efficiency and accuracy of NLP models.

## Question 5: Distributional Hypothesis

The Distributional Hypothesis states that words that appear in similar contexts tend to have similar meanings. In other words, a word’s meaning can be inferred from the words that commonly surround it. This principle forms the foundation of many NLP techniques, including word embeddings like Word2Vec and GloVe, where semantic relationships are learned based on contextual co-occurrence patterns. For example, if the words “cat” and “dog” frequently appear in similar linguistic contexts, the model will learn that they are semantically related.

## Question 6: Emergent Behaviours in LLMs

Emergent behaviors refer to complex capabilities that arise in large language models as they scale in size and training data, even though these abilities were not explicitly programmed or directly trained. These behaviors appear only when the model reaches a sufficient level of complexity. An example of emergent behavior is multi-step reasoning, where a large language model can solve logical or mathematical problems by generating a sequence of intermediate reasoning steps, even if it was not explicitly trained on formal reasoning tasks. This demonstrates that advanced capabilities can spontaneously arise from increased model scale.

## Question 7: How Self-Attention Links "it" to "the tiger"

In the sentence “The tiger jumped out of a tree… because it was thirsty,” self-attention enables the model to determine that the pronoun “it” refers to “the tiger.” Self-attention works by allowing each word in the sentence to evaluate its relationship with every other word. When processing the word “it,” the model assigns higher attention weights to relevant earlier words in the sentence, such as “tiger,” based on contextual similarity and grammatical structure. This mechanism allows the model to correctly resolve references and understand that “it” is not referring to the tree, but to the tiger, preserving semantic coherence across the sentence.